

Arrays, Multidimensional Arrays & Pointers

Q. Define Arrays. Explain one-dimensional arrays, multi-dimensional arrays, and the concept of an array of pointers with syntax and examples.

> **Definition to Remember**

1. One-Dimensional (1D) Array

- Simplest form a single list of elements.

```c

## 2. Multi-Dimensional Arrays

- An array of arrays. Most common: **2D Array** (matrix rows and columns).

```c

3. Array of Pointers

- An array where each element is a **pointer** (holds a memory address).

```c

4. Key Properties of Arrays

| Property | Detail |

> **Must-Write Points (for 10 marks)**

> **Quick Recall**

Algorithmic Implementations: 2D Arrays & Sparse Matrix (C Code)

1. Matrix Multiplication Algorithm & C Code

```
``c
#include <stdio.h>
#define R1 2
#define C1 3
#define R2 3
#define C2 3

void multiplyMatrices(int A[R1][C1], int B[R2][C2], int C[R1][C2]) {
 for (int i = 0; i < R1; i++)
 for (int j = 0; j < C2; j++)
 C[i][j] = 0;

 for (int i = 0; i < R1; i++)
 for (int k = 0; k < C1; k++)
 for (int l = 0; l < C2; l++)
 C[i][l] += A[i][k] * B[k][l];
}
```

int main() {

```
multiplyMatrices(A, B, C);
printf("--- Resultant Product Matrix (2x2) ---\n");
```

```
for (int
```

```

```

## 2. Sparse Matrix Triplet Representation (C Code)

```
```c
```

```
#includ
```

```
struct Element {
```

```
int row
```

```
struct Sparse {
```

```
int m;
```

```
void createSparse(struct Sparse *s, int matrix[4][4]) {
```

```
s->m =
```

```
for (int i = 0; i < 4; i++) {
```

```
for (int
```

```
void displaySparse(struct Sparse s) {
```

```
printf("
```

```
int main() {
```

```
int main()
```

```
struct Sparse s;
```

```
create
```

```
printf("--- Sparse Matrix Triplet Representation ---\n");
```

```
display
```

```
return 0;
```

```
}
```